

1. Introduction

RideSharingDatabase is a relational database designed to handle the core logistics of a ride-sharing service. The system manages users, user roles, cars, addresses, and the life cycle of a ride request.

2. Table Definitions

User & Role Management

- **roles:** Defines system permissions (admin, passenger, driver).
- **persons:** Stores core user data like names, email, birthday, billing address, etc.
- **personToRole:** A junction table allowing users to hold multiple roles. (A person can take both passenger and driver roles simultaneously).

Geographic Data

- **countries & cities:** A hierarchical structure ensuring standardized location naming.
- **addresses:** Stores specific location details (street, floor, postal code) for use in billing or ride routing.
- **personsToAddresses:** A many-to-many link allowing users to save multiple locations (e.g., "Home," "Work").

Vehicle Management

- **carManufacturers & carModels:** Separates brand from model to prevent data entry errors and simplifies filtering.
- **cars:** Stores specific vehicle details like license plates and equipment (racks, hooks, number of passengers it can take, etc.).
- **carsToDrivers:** A many-to-many relationship that links drivers to cars. One driver can have several cars, and a car can be used by several drivers.

Ride Lifecycle

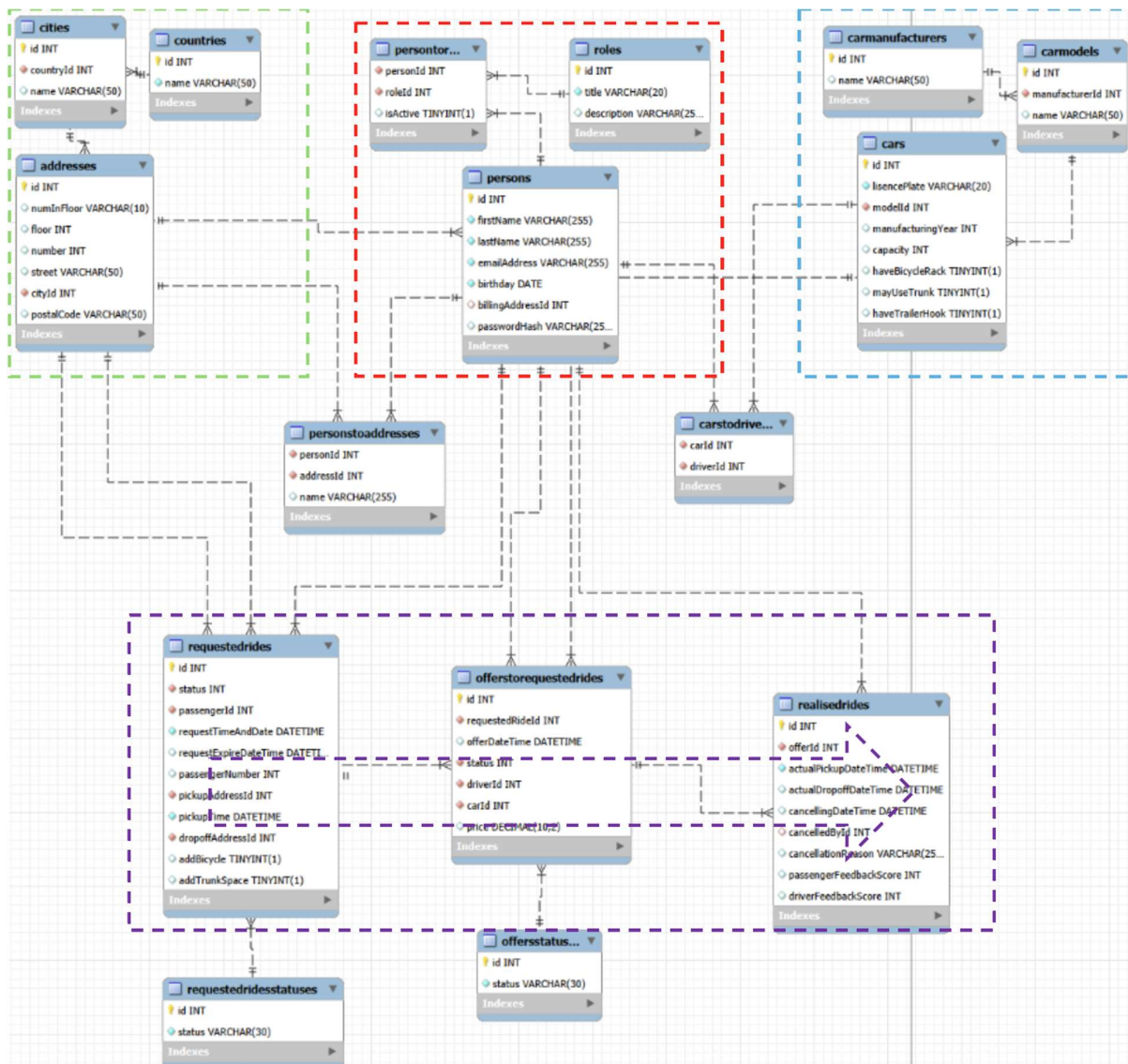
- **requestedRidesStatuses & requestedRides:** Manages the initial passenger request, including pickup/drop-off points and expiration logic.
- **offersStatuses & offersToRequestedRides:** Tracks driver bids on specific rides, including the proposed price and car used.
- **realisedRides:** Records the final "trip" data, including actual timing, feedback scores, and cancellation details.

3. Key Business Rules

- A ride must have a pickup and drop-off location.
- Drivers must be linked to a specific car to make an offer.
- Ride Requests are automatically expired via database events if no agreement made between a passenger and a driver before the time runs off.

3. Database Diagram:

Below, one can see the database diagram. The red square shows the tables that hold persons' data, the green square shows the tables that store addresses, and the blue square shows tables related to cars' data. The dark purple square shows the tables that store a ride request life cycle tables and the arrow shows the flow of a ride request, from register, to making an offer to a request and finally, realizing a ride.



4. Automation & Data Integrity

The following automation tools are developed to handle special scenarios.

Database Events: Automated Expiry

When a request for a ride is registered, the passenger must enter an expire date-time which indicates until when the passenger expects to receive an offer for the request and if no offer is made by then, the passenger has the chance to find another means. To handle expired rides, a scheduled event, “**updateExpiredRideRequests**” is developed which runs every **60 seconds**. This event scans the requestedRides table for any "Pending" requests where the requestExpireDateTime is in the past and automatically updates their status to "Expired".

Stored Procedures: Transactions

The AcceptRideOffer procedure handles the complex transition that occurs when a driver's offer is accepted. To prevent data inconsistency (executing only part of the steps required for an update) this procedure wraps multiple steps into a single **Transaction**:

1. It updates the chosen offer to "Accepted."
2. It sets the main ride request status to "Accepted."
3. It automatically marks all other competing offers for that ride as "Expired."
4. It initializes the record in realisedRides.